SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering
ASSESSMENT TOOL 1 – MINI APPLICATION DEVELOPMENT (CHATBOT / SUMMARIZER / TRANSLATOR)

Course Code:	CSA65	**Course Title:**	Generative AI and Large Language Models
CO Assessed:	CO4 – Precision (BL3, BL4,BL5)	**Assessment:**	Assessment Tool 1-Mini Application Development (Chatbot / Summarizer / Translator)
Weightage:	20% of CIA	**Total Marks:**	2 * 50= 100
CO4 Statement:	*Design and implement multimodal Generative AI applications integrating text, image, and speech models for engineering applications.*(BL3, BL4,BL5)		
Student Name:	A. Mounish	**Reg. No.:**	192472273
Section / Batch:	CSE AI	**Date:**	20/08/26

Instructions to Students

- **Answer any 2 questions. Each question carries 50 marks. Total: 100 marks.**
- Implement the solution using **Python**.
- Select and justify an appropriate AI/LLM model or technique.
- Develop a **working application with a user interface**.
- Test the application using multiple input cases.
- Demonstrate handling of invalid, empty, or unexpected inputs.
- Clearly explain the application architecture and workflow.
- Submit the source code, test results, screenshots/demo, and documentation.
- Explain the limitations of the selected model/technique and suggest possible improvements.

Code of Conduct

*I **A. Mounish / 192472273** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the Student: _____

SECTION A – MINI APPLICATION DEVELOPMENT (CHATBOT / SUMMARIZER / TRANSLATOR)

TEXT SUMMARIZATION APPLICATIONS

15. Meeting Minutes Summarizer – Develop an AI application that summarizes meeting transcripts and identifies important decisions, discussions, and action items.

1. Question Requirement

Question 18 requires the development of a Meeting Minutes Summarizer. The application must summarize meeting transcripts and identify important decisions, discussions, and action items. The implemented mini application accepts a meeting transcript through a Streamlit interface and produces these structured outputs.

The assessment source places Question 18 under Text Summarization Applications and defines the task specifically as an AI application for meeting transcripts, with emphasis on decisions, discussions, and action items. □filecite□turn1file1□L89-L100□

2. Abstract

The Meeting Minutes Summarizer is a Python-based AI mini application designed to convert meeting transcripts into concise and organized meeting minutes. The system uses an extractive summarization approach based on TF-IDF sentence scoring. Sentences are represented using TF-IDF values, ranked according to their overall importance, and the highest-scoring sentences are selected to form the summary.

In addition to the summary, the application identifies important decisions, discussions, and action items using targeted linguistic keywords and patterns. A Streamlit interface provides a simple browser-based environment in which users can paste a transcript, choose the desired summary length, and generate structured meeting minutes. The implementation was tested with normal meeting transcripts, empty input, unexpected content, and different summary-length settings.

3. Introduction

Meeting discussions often contain a mixture of explanations, decisions, responsibilities, deadlines, and follow-up activities. Reading an entire transcript to identify the most important information can be time-consuming. An automated summarization application can reduce this effort by extracting the most informative sentences and organizing key meeting outcomes into clearly labelled sections.

This application focuses on a controlled meeting-minutes use case. Instead of generating unrestricted text, it combines extractive sentence scoring with rule-based identification of decisions, discussions, and action items. This makes the result transparent and easy to demonstrate in a mini-application environment.

4. Objectives

- Accept a meeting transcript through a user-friendly interface.
- Generate a concise summary of the meeting.
- Identify important decisions made during the meeting.
- Identify important discussions or topics discussed.
- Identify action items and assigned follow-up tasks.
- Allow the user to control the approximate number of summary sentences.
- Handle empty input safely.
- Handle unexpected or non-meeting content without crashing.

- Provide a clear and professional browser-based output.

5. Problem Identification

Manual preparation of meeting minutes requires a person to review the transcript, distinguish important information from routine conversation, and record decisions and assigned tasks. This becomes increasingly difficult as transcript length increases. The problem addressed by the proposed system is the automatic extraction of concise, useful meeting information from an unstructured transcript.

6. Proposed Solution

The proposed solution uses a Streamlit interface connected to a Python summarization module. The transcript is first validated. When valid text is provided, the system divides it into sentences and calculates TF-IDF representations. Sentence scores are obtained by summing TF-IDF weights, and the highest-scoring sentences are selected according to the user's requested summary length.

At the same time, the transcript is scanned for decision-related, discussion-related, and action-related terms. Matching sentences are placed into the corresponding output sections. The final interface displays the meeting summary, important decisions, action items, and important discussions.

7. AI Technique Used – TF-IDF Sentence Scoring

The implemented summarization technique is extractive summarization using TF-IDF sentence scoring. TF-IDF assigns greater importance to words that are useful for distinguishing sentences within the transcript. The sentence vector is then used to estimate the information content of each sentence.

The application calculates a score for every sentence by summing its TF-IDF weights. Sentences with higher scores are treated as more informative. The top-ranked sentences are then restored to their original transcript order so that the generated summary remains readable and logically connected.

This technique is suitable for the mini application because it is lightweight, deterministic, easy to explain, and does not require a large language model API. It also provides a transparent basis for demonstrating extractive summarization.

8. System Architecture

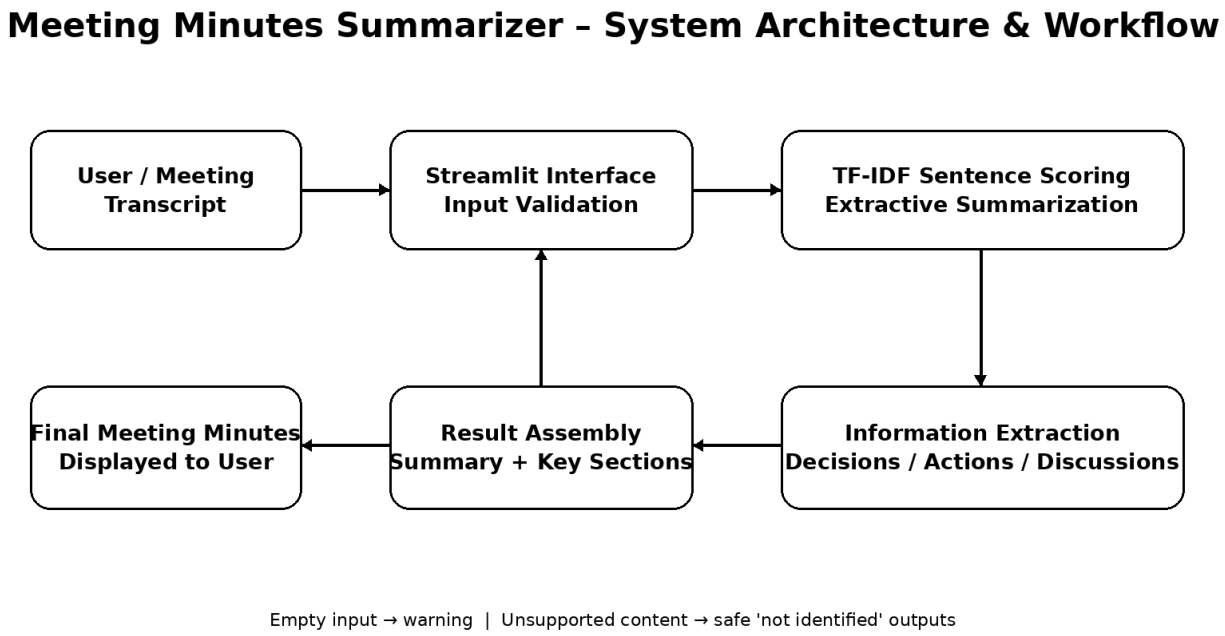

Figure 1. System architecture and workflow of the Meeting Minutes Summarizer.

The architecture contains five main processing stages: user input, Streamlit interface and validation, TF-IDF sentence scoring, structured information extraction, and result assembly. The final output is displayed in separate sections so that users can quickly identify the meeting summary, decisions, action items, and discussions.

9. Detailed Workflow

Step 1: The user opens the Streamlit Meeting Minutes Summarizer.

Step 2: The user pastes a meeting transcript into the transcript input area.

Step 3: The user selects the desired summary length using the slider.

Step 4: The application checks whether the transcript is empty.

Step 5: The transcript is split into individual sentences.

Step 6: TF-IDF vectors are generated for the sentences.

Step 7: Each sentence receives an importance score based on its TF-IDF weights.

Step 8: The highest-scoring sentences are selected according to the requested summary length.

Step 9: The selected sentences are reordered into their original sequence to form the summary.

Step 10: Decision-related keywords are used to identify important decisions.

Step 11: Action-related keywords are used to identify action items.

Step 12: Discussion-related keywords are used to identify important discussions.

Step 13: The application displays all results in a structured meeting-minutes format.

10. Implementation Details

10.1 summarizer.py

The summarizer.py module contains the core processing functions. split_sentences() separates the transcript into sentences. generate_summary() applies TfidfVectorizer, calculates sentence scores, selects the required number of sentences, and creates the final extractive summary. Separate functions identify decisions, action items, and discussions using predefined keyword patterns.

10.2 app.py

The app.py module provides the Streamlit interface. It displays the application title and description, accepts the meeting transcript through a text area, provides a Summary Length slider, and uses the Generate Meeting Minutes button to trigger processing. The output is organized into four sections: Meeting Summary, Important Decisions, Action Items, and Important Discussions.

10.3 meeting_data.txt

A sample meeting transcript was prepared for demonstration and testing. It contains project discussions, decisions, deadlines, and assigned action items so that every required output category can be demonstrated.

10.4 requirements.txt

The application uses Streamlit for the interface and scikit-learn for TF-IDF vectorization. These dependencies provide the basic environment required to run the implementation.

11. User Interface and Demonstration

The screenshots below are the actual implementation evidence supplied during the development of Q18. They are placed at the relevant points to demonstrate the interface, transcript input, generated output, robustness tests, and summary-length control.

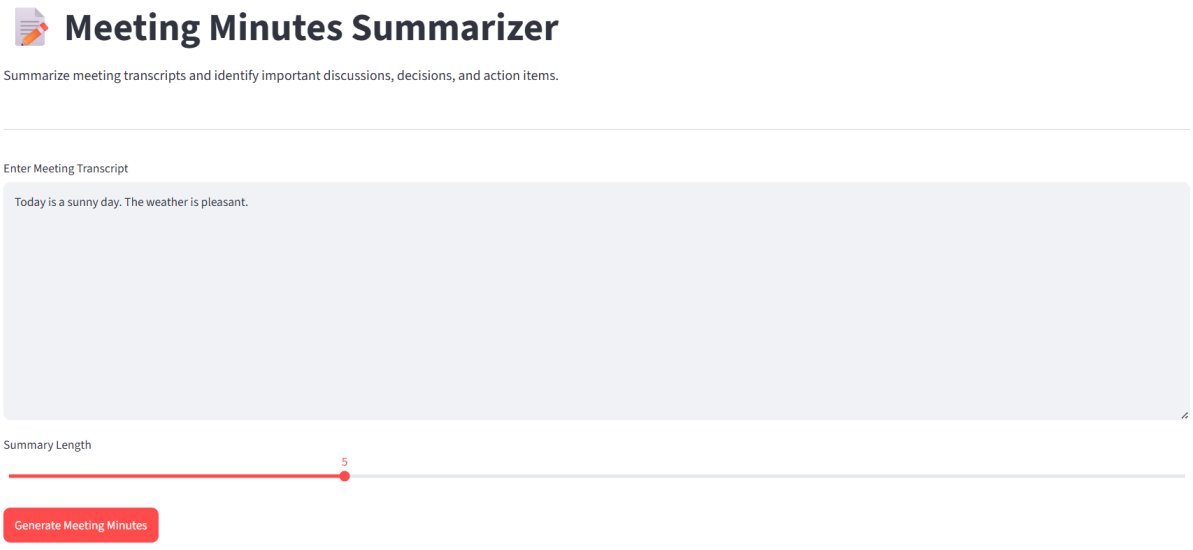

Figure 2. Initial Meeting Minutes Summarizer interface.

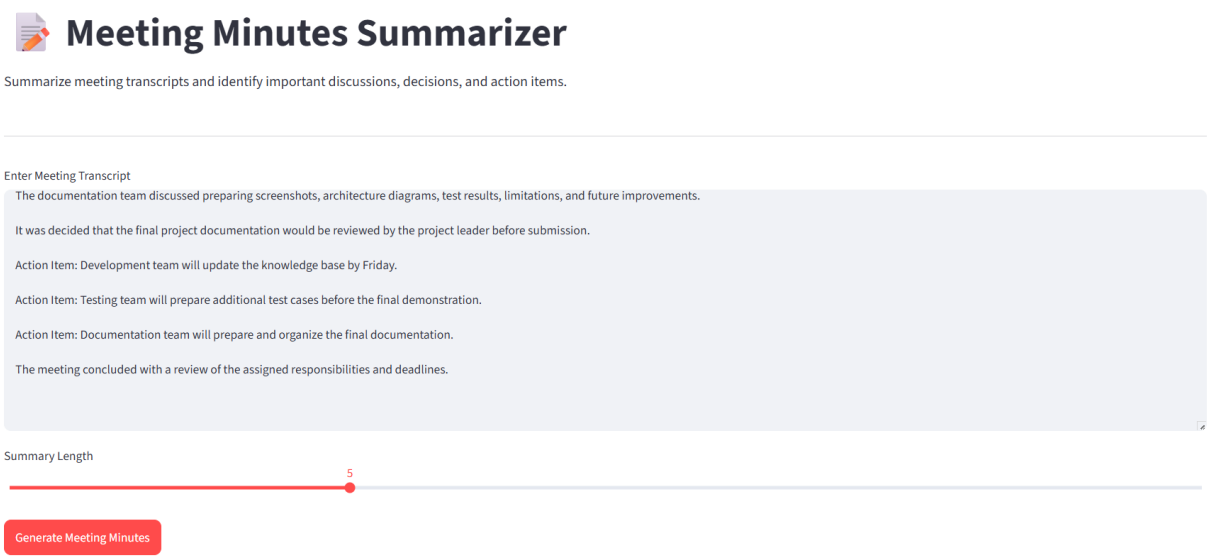

Figure 3. Meeting transcript entered before generating minutes.

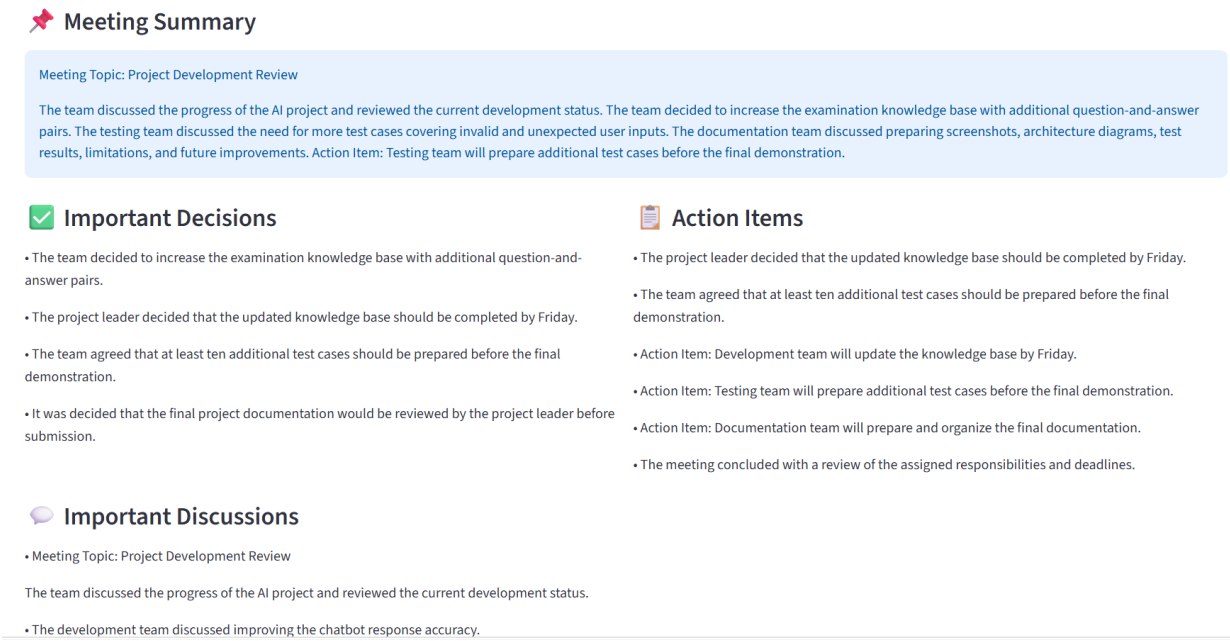

Figure 4. Generated summary with decisions, action items, and discussions.

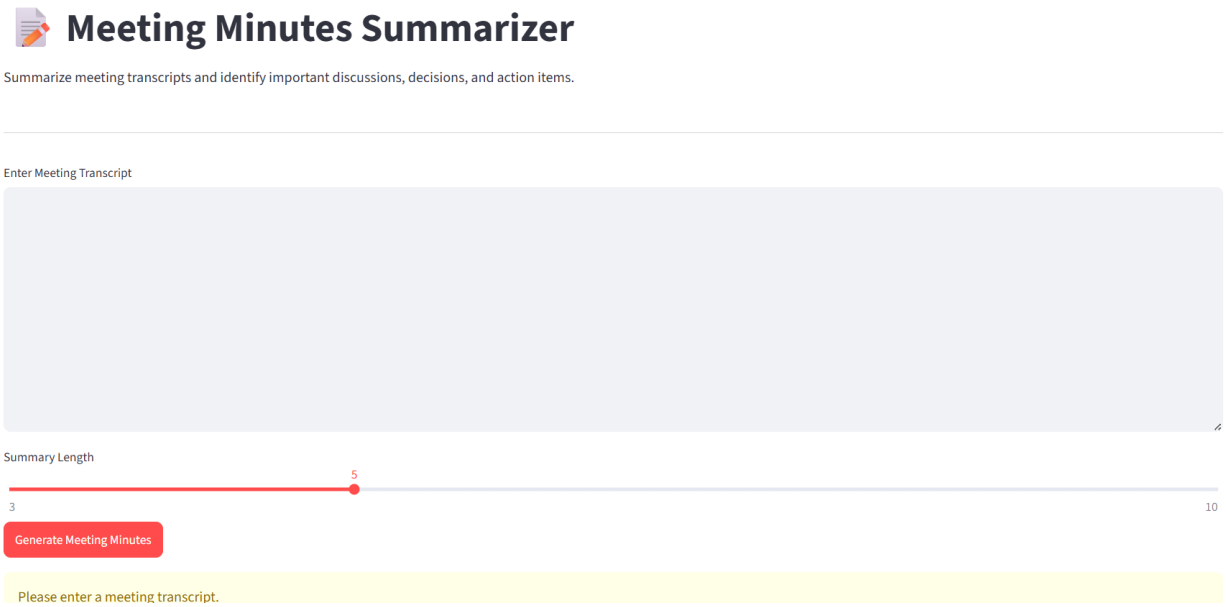

Figure 5. Empty-input validation message.

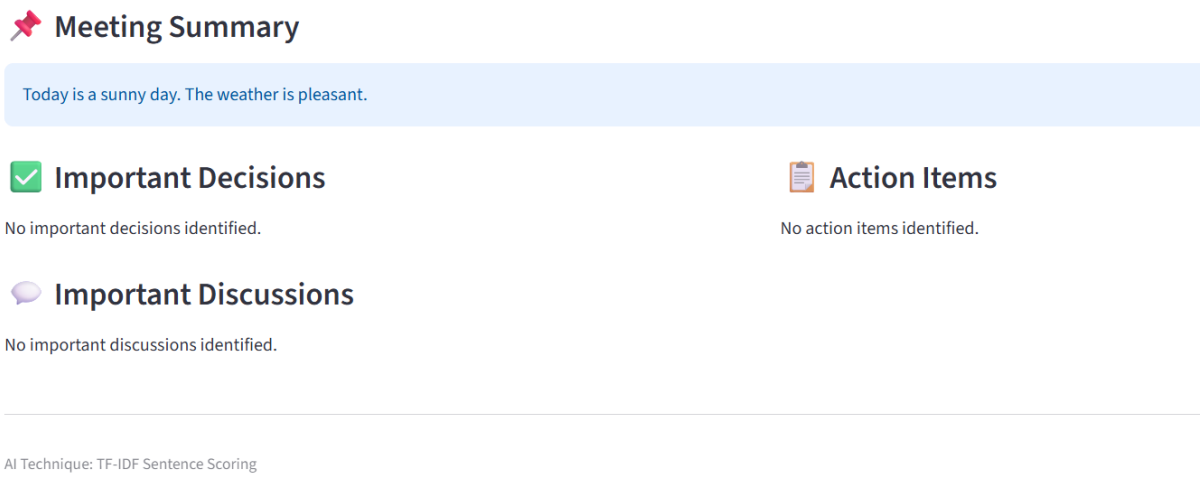

Figure 6. Unexpected/non-meeting input handled without application failure.

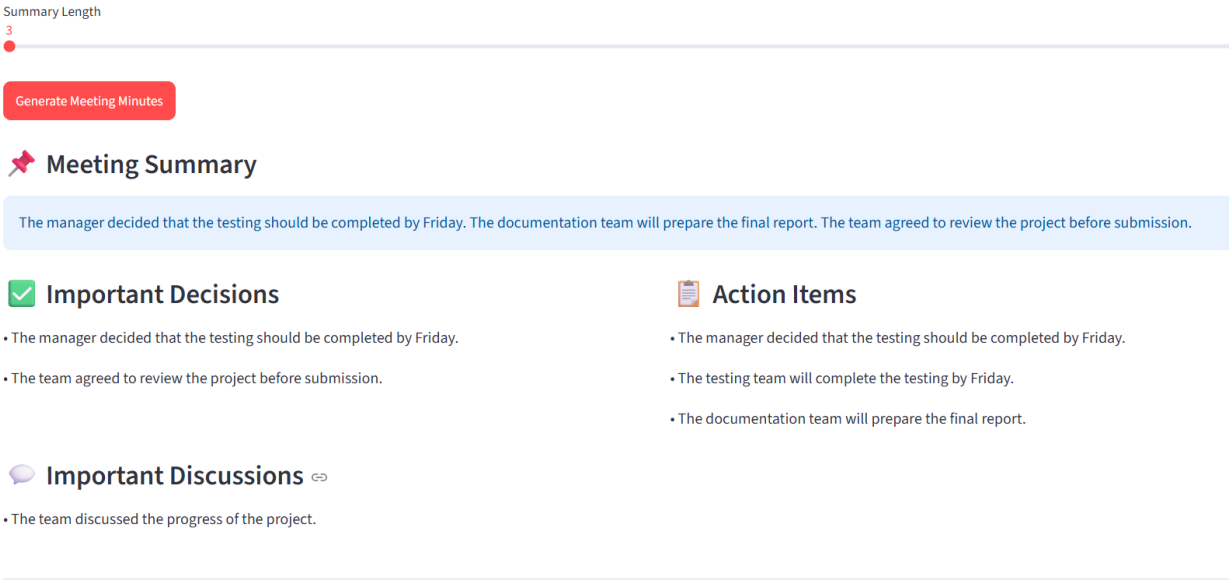

Figure 7. Weekly Team Meeting result with shorter summary setting.

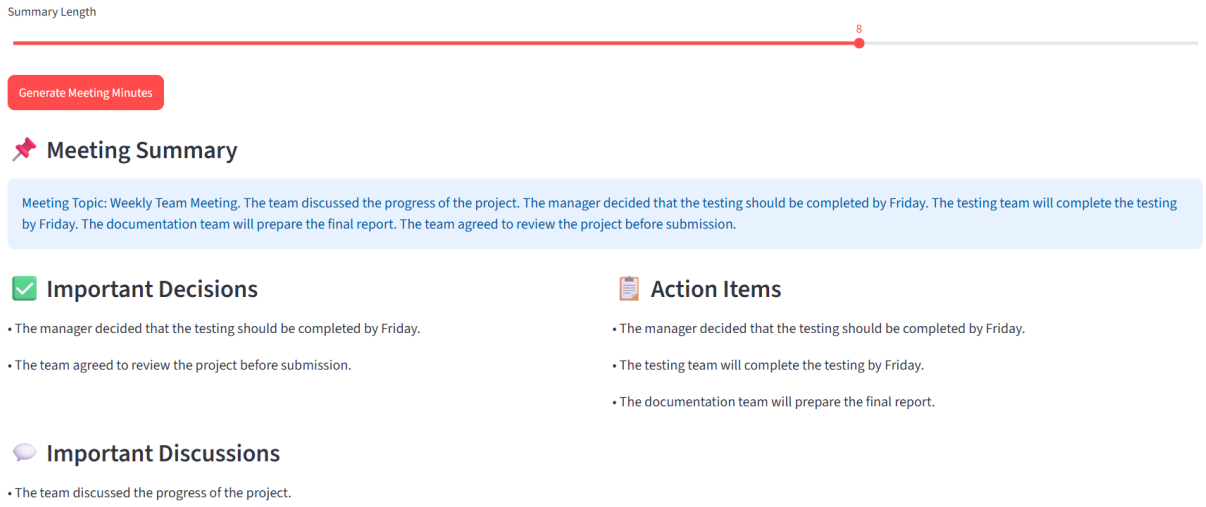

Figure 8. Weekly Team Meeting result with longer summary setting.

12. Functional Output Explanation

12.1 Meeting Summary

The Meeting Summary section presents the selected high-scoring sentences from the transcript. The summary is extractive, meaning the system selects important sentences from the original transcript rather than generating completely new sentences.

12.2 Important Decisions

The Important Decisions section identifies sentences containing decision-oriented expressions such as 'decided', 'agreed', 'approved', and related phrases. In the demonstrated meeting, the system successfully identified decisions concerning testing deadlines and project review.

12.3 Action Items

The Action Items section identifies sentences containing task-oriented patterns such as 'Action Item', 'will', 'should', 'assigned', or 'responsible'. The demonstrated results include tasks assigned to the testing and documentation teams.

12.4 Important Discussions

The Important Discussions section identifies sentences containing discussion-oriented expressions such as 'discussed', 'discussion', 'reviewed', 'talked', and 'considered'. This allows the user to see the main topics discussed in the meeting.

13. Testing Strategy

Testing was performed using functional, robustness, and parameter-control cases. The functional tests verified that the application could summarize a meeting and identify decisions, action items, and discussions. Robustness tests verified empty input and unexpected content. Parameter testing verified that changing the Summary Length value affected the amount of selected summary content.

14. Test Cases and Results

Test ID	Test Input	Expected Behaviour	Result
TC01	Full project-development meeting transcript	Summary and all three key sections generated	Pass
TC02	Weekly Team Meeting transcript	Summary, decisions, action items and discussions identified	Pass
TC03	Empty transcript	Warning: Please enter a meeting transcript.	Pass
TC04	Non-meeting/weather text	Application runs; no decisions/actions/discussions identified	Pass
TC05	Summary Length = 3	Shorter extractive summary generated	Pass
TC06	Summary Length = 8	Longer extractive summary generated	Pass

Table 1. Functional, robustness, and summary-length test cases.

15. Test Evidence Summary

The supplied screenshots demonstrate that the application successfully produced structured outputs for the full project-development transcript and the weekly team meeting transcript. The empty-input screenshot shows that the application prevents processing when no transcript is entered. The unexpected-content test shows that the application does not crash when the text does not contain meeting information and instead reports that no important decisions, action items, or discussions were identified.

The two summary-length demonstrations show the same weekly meeting processed with Summary Length values of 3 and 8. This provides direct evidence that the user can control the approximate amount of extractive summary content.

16. Advantages

- Simple and user-friendly Streamlit interface.
- Fast and lightweight extractive summarization.
- No external API or large model dependency is required for the demonstrated implementation.
- Structured separation of summary, decisions, action items, and discussions.
- User-controlled summary length.
- Clear handling of empty input.
- Safe handling of non-meeting content without application failure.
- Easy-to-understand implementation suitable for academic demonstration.

17. Limitations

- TF-IDF is based on word importance and does not provide deep semantic understanding.
- The summary is extractive and therefore selects existing sentences rather than generating a fully rewritten abstract.
- Decision, action-item, and discussion extraction depends on predefined keywords and patterns.
- The current implementation does not identify speaker names, owners, dates, or deadlines as separate structured fields.
- Very long or poorly punctuated transcripts may reduce sentence-splitting and scoring quality.
- The application does not automatically ingest audio recordings or speech-to-text output.

18. Future Improvements

- Use transformer-based embeddings or an LLM for stronger semantic summarization.
- Add abstractive summarization alongside the current extractive approach.
- Automatically identify action-item owners, deadlines, and task status.
- Add speaker diarization and speaker-aware meeting minutes.
- Accept meeting audio and perform speech-to-text before summarization.
- Add PDF, DOCX, and TXT transcript upload support.
- Provide export options for generated minutes in PDF or Word format.
- Add multilingual meeting summarization.

19. Conclusion

The Meeting Minutes Summarizer successfully implements the core requirement of Question 18 by accepting meeting transcripts, generating concise summaries, and identifying important decisions, discussions, and action items. The application uses TF-IDF sentence scoring for transparent extractive summarization and keyword-based extraction for structured meeting information.

The supplied implementation screenshots demonstrate the working interface, successful functional outputs, empty-input handling, unexpected-input handling, and summary-length control. The solution is lightweight and appropriate for a mini application, while the listed improvements provide a clear path toward a more advanced LLM-based meeting assistant.

20. Requirements Mapping

Requirement	Implementation Evidence
Meeting transcript summarization	Implemented using TF-IDF sentence scoring.
Important decisions	Extracted using decision-oriented keywords and patterns.
Important discussions	Extracted using discussion-oriented keywords and patterns.
Action items	Extracted using action-oriented keywords and patterns.
Working AI application	Python implementation with Streamlit interface.
Multiple test cases	Functional, robustness, and parameter-control tests documented.
Empty input handling	Validation warning demonstrated.
Unexpected input handling	Non-meeting text processed safely without application failure.
Summary length control	Slider tested at values 3 and 8.
Documentation evidence	Architecture, workflow, screenshots, test table, limitations and future improvements included.

21. References

1. Assessment Tool 1 – Mini Application Development, SIMATS Engineering, Department of Computer Science and Engineering. Question 18: Meeting Minutes Summarizer. The source defines the task as an AI application that summarizes meeting transcripts and identifies important decisions, discussions, and action items. □filecite□turn1file1□L89-L100□

2. Scikit-learn implementation concepts used in the application: TfidfVectorizer and TF-IDF-based text representation.

3. Streamlit implementation concepts used in the application: text areas, sliders, buttons, columns, status messages, and output components.